

# 06 — So sánh GAN 3D: Normalized vs Unnormalized

**Dự án:** Latent Manipulation of Brain MRI using Volume-Preserving GANs

**Mục tiêu:** So sánh 2 model từ file 04 và 05, chọn model tốt hơn để dùng cho latent manipulation.

**Metrics:**

- **Loss\_G** — loss Generator tốt nhất (thấp hơn = tốt hơn)
- **SSIM** — độ giữ nguyên cấu trúc giải phẫu (cao hơn = tốt hơn)

**Output:**

```
compare_3d/  
└─ comparison_result.json
```

## Bước 1: Cấu hình

```
import os  
  
# ==== ĐƯỜNG DẪN 2 MODEL ====  
MODEL_NORM_PATH =  
'/kaggle/input/datasets/cminhnguyndsd/sds/conditional-gan3d-  
normalized/best_model.pth'  
MODEL_UNNORM_PATH = '/kaggle/input/datasets/dyio147/conditional-gan3d-  
unnormalized/best_model.pth'  
  
# ==== DATA ====  
DATA_NORM_DIR = '/kaggle/input/datasets/minhbodoi/3000-preprocessed-  
3d/preprocessed_3d/normalized'  
DATA_UNNORM_DIR = '/kaggle/input/datasets/minhbodoi/3000-preprocessed-  
3d/preprocessed_3d/unnormalized'  
LABELS_CSV = '/kaggle/input/datasets/minhbodoi/3000-preprocessed-  
3d/preprocessed_3d/preprocessing_log.csv'  
  
OUTPUT_DIR = '/kaggle/working/compare_3d'  
os.makedirs(OUTPUT_DIR, exist_ok=True)  
  
LATENT_DIM = 256  
print('Cấu hình xong!')  
  
Cấu hình xong!
```

## Bước 2: Import thư viện

```
!pip install nibabel scikit-image -q

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import nibabel as nib
import numpy as np
import pandas as pd
import json
from skimage.metrics import structural_similarity as ssim
import warnings
warnings.filterwarnings('ignore')

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Device: {DEVICE}')

Device: cuda
```

## Bước 3: Load model

```
class AgeEmbedding(nn.Module):
    def __init__(self, embed_dim=256):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(1, 128), nn.ReLU(),
nn.Linear(128, embed_dim))
    def forward(self, age): return self.net(age.unsqueeze(-1))

# — Kiến trúc cũ (U-Net GAN) —————
class UNetBlock3D(nn.Module):
    def __init__(self, in_ch, out_ch, down=True, use_bn=True,
dropout=False):
        super().__init__()
        layers = []
        if down: layers.append(nn.Conv3d(in_ch, out_ch, 4, 2, 1,
bias=False))
        else: layers.append(nn.ConvTranspose3d(in_ch, out_ch, 4, 2,
1, bias=False))
        if use_bn: layers.append(nn.BatchNorm3d(out_ch))
        if dropout: layers.append(nn.Dropout(0.5))
        layers.append(nn.LeakyReLU(0.2) if down else nn.ReLU())
        self.block = nn.Sequential(*layers)
    def forward(self, x): return self.block(x)

class Generator3D(nn.Module):
    def __init__(self, latent_dim=256):
        super().__init__()
```

```

        self.age_embed = AgeEmbedding(latent_dim)
        self.age_proj = nn.Linear(latent_dim, 256)
        self.e1 = UNetBlock3D(1, 32, down=True, use_bn=False)
        self.e2 = UNetBlock3D(32, 64, down=True)
        self.e3 = UNetBlock3D(64, 128, down=True)
        self.e4 = UNetBlock3D(128, 256, down=True, use_bn=False)
        self.d1 = UNetBlock3D(256, 128, down=False, dropout=True)
        self.d2 = UNetBlock3D(256, 64, down=False)
        self.d3 = UNetBlock3D(128, 32, down=False)
        self.out = nn.Sequential(nn.ConvTranspose3d(64, 1, 4, 2, 1),
nn.Tanh())
    def forward(self, x, age):
        e1=self.e1(x); e2=self.e2(e1); e3=self.e3(e2); e4=self.e4(e3)
        z = e4 + self.age_proj(self.age_embed(age)).view(-1, 256, 1,
1, 1)
        d1=self.d1(z); d2=self.d2(torch.cat([d1,e3],1));
d3=self.d3(torch.cat([d2,e2],1))
        return self.out(torch.cat([d3,e1],1))

# — Kiến trúc mới (StyleGAN-inspired với AdaIN) —————
class MappingNetwork(nn.Module):
    def __init__(self, latent_dim=256, w_dim=256, n_layers=4):
        super().__init__()
        layers = [AgeEmbedding(latent_dim), nn.ReLU()]
        in_dim = latent_dim
        for _ in range(n_layers - 1):
            layers += [nn.Linear(in_dim, w_dim), nn.ReLU()]
            in_dim = w_dim
        self.net = nn.Sequential(*layers)
        self.out = nn.Linear(in_dim, w_dim)
    def forward(self, age): return self.out(self.net(age))

class AdaIN3D(nn.Module):
    def __init__(self, channels, w_dim=256):
        super().__init__()
        self.norm = nn.InstanceNorm3d(channels, affine=False)
        self.style = nn.Linear(w_dim, channels * 2)
    def forward(self, x, w):
        style = self.style(w).unsqueeze(-1).unsqueeze(-1).unsqueeze(-
1)

        scale, shift = style.chunk(2, dim=1)
        return scale * self.norm(x) + shift

class Generator3D_StyleGAN(nn.Module):
    def __init__(self, latent_dim=256, w_dim=256):
        super().__init__()
        self.mapping = MappingNetwork(latent_dim, w_dim)
        self.e1 = UNetBlock3D(1, 32, down=True, use_bn=False)
        self.e2 = UNetBlock3D(32, 64, down=True)

```

```

        self.e3 = UNetBlock3D(64, 128, down=True)
        self.e4 = UNetBlock3D(128, 256, down=True, use_bn=False)
        self.d1 = nn.Sequential(nn.ConvTranspose3d(256, 128, 4, 2, 1,
bias=False), nn.Dropout(0.5), nn.ReLU())
        self.d2 = nn.Sequential(nn.ConvTranspose3d(256, 64, 4, 2, 1,
bias=False), nn.ReLU())
        self.d3 = nn.Sequential(nn.ConvTranspose3d(128, 32, 4, 2, 1,
bias=False), nn.ReLU())
        self.out = nn.Sequential(nn.ConvTranspose3d(64, 1, 4, 2, 1),
nn.Tanh())
        self.adain1 = AdaIN3D(128, w_dim)
        self.adain2 = AdaIN3D(64, w_dim)
        self.adain3 = AdaIN3D(32, w_dim)
    def forward(self, x, age):
        w = self.mapping(age)
        e1=self.e1(x); e2=self.e2(e1); e3=self.e3(e2); e4=self.e4(e3)
        d1=self.adain1(self.d1(e4), w)
        d2=self.adain2(self.d2(torch.cat([d1,e3],1)), w)
        d3=self.adain3(self.d3(torch.cat([d2,e2],1)), w)
        return self.out(torch.cat([d3,e1],1))

```

```

# — Kiến trúc Concat 3D (age_fuse) —————
class Generator3D_Concat(nn.Module):
    def __init__(self, latent_dim=256):
        super().__init__()
        self.age_embed = AgeEmbedding(latent_dim)
        self.age_proj = nn.Linear(latent_dim, 256)
        self.age_fuse = nn.Conv3d(512, 256, 1)
        self.e1=UNetBlock3D(1,32,down=True,use_bn=False);
self.e2=UNetBlock3D(32,64,down=True)
        self.e3=UNetBlock3D(64,128,down=True);
self.e4=UNetBlock3D(128,256,down=True,use_bn=False)
        self.d1=UNetBlock3D(256,128,down=False,dropout=True)
        self.d2=UNetBlock3D(256,64,down=False);
self.d3=UNetBlock3D(128,32,down=False)

self.out=nn.Sequential(nn.ConvTranspose3d(64,1,4,2,1),nn.Tanh())
    def forward(self, x, age):
        e1=self.e1(x); e2=self.e2(e1); e3=self.e3(e2); e4=self.e4(e3)
        age_feat=self.age_proj(self.age_embed(age)).view(-
1,256,1,1,1).expand_as(e4)
        z=self.age_fuse(torch.cat([e4,age_feat],dim=1))
        d1=self.d1(z); d2=self.d2(torch.cat([d1,e3],1));
d3=self.d3(torch.cat([d2,e2],1))
        return self.out(torch.cat([d3,e1],1))

def load_generator(ckpt_path):

```

```

ckpt = torch.load(ckpt_path, map_location=DEVICE)
keys = list(ckpt['G_state'].keys())
if any('mapping' in k for k in keys):
    G = Generator3D_StyleGAN(LATENT_DIM).to(DEVICE)
    print(' -> StyleGAN architecture detected')
elif any('age_fuse' in k for k in keys):
    G = Generator3D_Concat(LATENT_DIM).to(DEVICE)
    print(' -> Conditional GAN Concat architecture detected')
else:
    G = Generator3D(LATENT_DIM).to(DEVICE)
    print(' -> U-Net architecture detected')
G.load_state_dict(ckpt['G_state'])
G.eval()
return G, ckpt

print('Loading Normalized model...')
G_norm, ckpt_norm = load_generator(MODEL_NORM_PATH)
print('Loading Unnormalized model...')
G_unnorm, ckpt_unnorm = load_generator(MODEL_UNNORM_PATH)

print(f'Normalized - epoch: {ckpt_norm["epoch"]} | best_val_SSIM: {ckpt_norm.get("best_val_ssim", -1):.4f}')
print(f'Unnormalized - epoch: {ckpt_unnorm["epoch"]} | best_val_SSIM: {ckpt_unnorm.get("best_val_ssim", -1):.4f}')

Loading Normalized model...
-> Conditional GAN Concat architecture detected
Loading Unnormalized model...
-> Conditional GAN Concat architecture detected
Normalized - epoch: 13 | best_val_SSIM: 0.9734
Unnormalized - epoch: 23 | best_val_SSIM: 0.9805

```

## Bước 4: Dataset

```

def find_nii(data_dir, subject_id):
    for ext in ['.nii.gz', '.nii']:
        path = os.path.join(data_dir, f'{subject_id}{ext}')
        if os.path.exists(path):
            return path
    return None

class BrainMRI3DDataset(Dataset):
    def __init__(self, data_dir, labels_csv, age_min, age_max,
volume_size):
        self.age_min = age_min
        self.age_max = age_max
        self.volume_size = volume_size
        df = pd.read_csv(labels_csv)

```

```

        df['nii_path'] = df['subject_id'].apply(lambda x:
find_nii(data_dir, x))
        self.df = df[df['nii_path'].notna()].reset_index(drop=True)

        def normalize_age(self, age):
            return 2 * (age - self.age_min) / (self.age_max -
self.age_min) - 1

        def __len__(self): return len(self.df)

        def __getitem__(self, idx):
            row = self.df.iloc[idx]
            data =
nib.load(row['nii_path']).get_fdata().astype(np.float32)
            vol = torch.tensor(data).unsqueeze(0).unsqueeze(0)
            vol = F.interpolate(vol, size=(self.volume_size,) * 3,
                                mode='trilinear', align_corners=False)
            vol = vol.squeeze(0) * 2 - 1
            age_norm = torch.tensor(self.normalize_age(row['age']),
dtype=torch.float32)
            return vol, age_norm

loader_norm = DataLoader(
    BrainMRI3DDataset(DATA_NORM_DIR, LABELS_CSV,
                      ckpt_norm['age_min'], ckpt_norm['age_max'],
                      ckpt_norm.get('volume_size', 64)),
    batch_size=1, shuffle=False
)
loader_unnorm = DataLoader(
    BrainMRI3DDataset(DATA_UNNORM_DIR, LABELS_CSV,
                      ckpt_unnorm['age_min'], ckpt_unnorm['age_max'],
                      ckpt_unnorm.get('volume_size', 64)),
    batch_size=1, shuffle=False
)
print('Dataset sẵn sàng!')
Dataset sẵn sàng!

```

## Bước 5: Tính SSIM và so sánh

```

def compute_ssim_3d(G, loader, device):
    """
    Tính SSIM trung bình giữa volume real và fake.
    Tính SSIM trên từng axial slice rồi lấy trung bình.
    """
    scores = []
    G.eval()
    with torch.no_grad():
        for real_vols, ages_norm in loader:

```

```

        real_vols = real_vols.to(device)
        ages_norm = ages_norm.to(device)
        fake_vols = G(real_vols, ages_norm)
        real_np = (real_vols[0, 0].cpu().numpy() + 1) / 2
        fake_np = (fake_vols[0, 0].cpu().numpy() + 1) / 2
        slice_scores = [
            ssim(real_np[i], fake_np[i], data_range=1.0)
            for i in range(real_np.shape[0])
        ]
        scores.append(np.mean(slice_scores))
    return float(np.mean(scores))

print('Đang tính SSIM...')
ssim_norm = compute_ssim_3d(G_norm, loader_norm, DEVICE)
ssim_unnorm = compute_ssim_3d(G_unnorm, loader_unnorm, DEVICE)

# Châm điêm
score_norm = score_unnorm = 0
if ckpt_norm.get('best_val_ssim', 0) >
    ckpt_unnorm.get('best_val_ssim', 0): score_norm += 1
else:
    score_unnorm += 1
if ssim_norm > ssim_unnorm:
    score_norm += 1
else:
    score_unnorm += 1

winner = 'normalized' if score_norm >= score_unnorm else
'unnormalized'

print(f'\n{"Metric":<20} {"Normalized":>12} {"Unnormalized":>14}')
print('-' * 48)
print(f'{"best_val_SSIM":<20} {ckpt_norm.get("best_val_ssim",-1):>12.4f} {ckpt_unnorm.get("best_val_ssim",-1):>14.4f}')
print(f'{"SSIM":<20} {ssim_norm:>12.4f} {ssim_unnorm:>14.4f}')
print(f'{"Score":<20} {score_norm:>12}/2 {score_unnorm:>13}/2')
print(f'\n-> Model tot hon: {winner.upper()}')
print('-> Dung model nay cho Latent Manipulation!')

result = {
    'winner': winner,
    'normalized': {'best_val_ssim': ckpt_norm.get('best_val_ssim',-1), 'ssim': ssim_norm},
    'unnormalized': {'best_val_ssim':
        ckpt_unnorm.get('best_val_ssim',-1), 'ssim': ssim_unnorm},
}
with open(f'{OUTPUT_DIR}/comparison_result.json', 'w') as f:
    json.dump(result, f, indent=2)
print(f'\nKet qua luu tai: {OUTPUT_DIR}/comparison_result.json')

```

Đang tính SSIM...

| Metric        | Normalized | Unnormalized |
|---------------|------------|--------------|
| best_val_SSIM | 0.9734     | 0.9805       |
| SSIM          | 0.9732     | 0.9804       |
| Score         | 0/2        | 2/2          |

-> Model tốt hơn: UNNORMALIZED

-> Dùng model này cho Latent Manipulation!

Kết quả lưu tại: /kaggle/working/compare\_3d/comparison\_result.json

*# Tự động lưu kết quả thành Kaggle Dataset*

```
import json, os, subprocess
```

```
dataset_name = os.path.basename(OUTPUT_DIR).replace('_', '-')
kaggle_user = [l.split(':')[1].strip() for l in
                subprocess.run('kaggle config view', shell=True,
                                capture_output=True, text=True)
                .stdout.split('\n') if 'username' in l][0]
```

```
with open(f'{OUTPUT_DIR}/dataset-metadata.json', 'w') as f:
```

```
    json.dump({
        'title' : dataset_name,
        'id' : f'{kaggle_user}/{dataset_name}',
        'licenses': [{'name': 'CC0-1.0'}]
    }, f)
```

```
check = subprocess.run(f'kaggle datasets list --user {kaggle_user} --
search {dataset_name}',
                        shell=True, capture_output=True, text=True)
```

```
if dataset_name in check.stdout:
```

```
    os.system(f'kaggle datasets version -p {OUTPUT_DIR} -m "update"')
```

```
else:
```

```
    os.system(f'kaggle datasets create -p {OUTPUT_DIR}')
```

```
print(f'Done! {kaggle_user}/{dataset_name}')
```

Warning: Looks like you're using an outdated `kaggle` version  
(installed: 2.0.1), please consider upgrading to the latest version  
(2.0.2)

Starting upload for file comparison\_result.json

100%|██████████| 222/222 [00:00<00:00, 803B/s]

Upload successful: comparison\_result.json (222B)

Dataset version is being created. Please check progress at

<https://www.kaggle.com/datasets/minhbodoi/compare-3d>

Done! minhbodoi/compare-3d